Team Introduction

PS-023: Minimal Two-Node Cloud Hypervisor VM Controller

Team Disciples of GO Category: Cloud Computing | Hackathon: AIORI-3

MEMBER 1

Joy Samadder

Work Designation: Project Mentor / Faculty

University Position: Assistant Professor

Email: joy.samadder@makautwb.ac.in

MEMBER 2

Hrittik Biswas

Work Designation: Virtualization & Worker Agents

University Position: Student

Email: hrittikbiswas0101@gmail.com

MEMBER 3

Reek Saha

Work Designation: Controller & Scheduling

University Position: Student

Email: reeksaha938@gmail.com

Replace bracketed placeholders with the actual names and email addresses before submission.

Problem Description

Why a lightweight two-node VM controller is needed

- **Overhead:** Enterprise orchestration platforms like OpenStack or Kubernetes introduce unnecessary components (databases, message buses, multiple daemons) which are heavy for managing just two KVM hosts and a small number of VMs.
- **Orphaned processes:** If a worker or controller fails, VM processes, Unix sockets, TAP devices and disk copies can remain on the host, leaking resources.
- **Race conditions:** Concurrent VM creation requests can read the same “free memory” and overcommit a host, leading to resource exhaustion.
- **Resource limits:** A misbehaving guest can consume excessive CPU or memory unless limits are enforced at the host level (cgroups).
- **Partial failures:** If VM creation or startup fails, all temporary resources must be cleaned up and the allocation rolled back completely.
- **Security risks:** Without proper authentication, anyone who can access the API could start, stop or create VM processes on the hosts.

Our Objective

- **Minimal architecture:** one central controller and two worker agents (one per KVM host).
- **Authenticated REST API:** create, start, stop, delete and list VMs.
- **Memory-based placement:** select a suitable host based on available memory and reserve resources.
- **Resource enforcement:** apply CPU and memory limits at the host level.
- **Reliable cleanup:** prevent orphaned processes, sockets, TAPs and temporary disk files.

Explicit non-goals: live migration, distributed storage, browser VNC, image marketplace, and developing a new hypervisor.

Background

Technologies, architecture and design context

Virtualization with Linux KVM

Linux Kernel-based Virtual Machine (KVM) provides the virtualization foundation on each physical host. Both machines must support /dev/kvm and have networking configured before deployment.

Cloud Hypervisor

Cloud Hypervisor is the selected virtual machine monitor. The controller will use its management API to request VM lifecycle operations, while each worker communicates with the hypervisor running on its own host.

Central Controller and Worker Agents

The controller exposes a single management interface. Two worker agents run independently—one per host—and translate controller requests into local hypervisor operations. This separates coordination from host-level execution.

Memory-Aware Placement

When a VM is requested, the controller will collect host resource information and compare available memory with the requested VM memory. It will select a suitable host and track the allocation to reduce the risk of overcommitting memory.

Security and Resource Limits

Controller-to-worker requests will be authenticated. The controller and workers will validate VM resource requirements and enforce defined CPU and memory limits before launching a VM.

Reliability and Cleanup

The system must handle API errors, worker unavailability, and failed VM startup. Cleanup logic will remove VM processes and temporary resources where applicable, release reservations, and return a clear status to the client.

Design goal: A lightweight, authenticated and resource-aware VM management layer for two KVM hosts.

Literature & RFC Review

Existing interfaces and standards relevant to a lightweight, two-host VM controller. Bracketed numbers refer to the separate Bibliography slide (page 10).

Source / focus	What it establishes	Relevance to our design
[1] Cloud Hypervisor Management API Project API documentation	Documents VM lifecycle operations and the management interface used to configure, boot, inspect, and stop virtual machines.	The worker agent can call the local hypervisor API instead of implementing a new virtualization layer.
[2] Linux KVM API Linux kernel documentation	Describes the kernel interface that userspace virtualization software uses to create and manage virtual machines.	Supports the assumption that both worker hosts must be KVM-capable and have the required host configuration.
[3] Cloud Hypervisor project documentation Architecture and implementation docs	Provides implementation and operational context for running Cloud Hypervisor and integrating it with host resources.	Informs the agent responsibilities: validate local resources, invoke the hypervisor, report state, and clean up failures.
[4] RFC 6750 — Bearer Token Usage IETF security standard	Defines how bearer tokens are used to authorize access to protected HTTP resources, including the Authorization header pattern.	Provides a standards-based reference for authenticating controller requests and protecting worker API calls; deployment still needs secure token handling and transport.

Proposed Solution

Controller + two Worker Agents with a deterministic VM state machine

Topology: one Controller and two Worker Agents (one per KVM host). Users call an HTTPS REST API with a Bearer key.

Stack: Python 3.11, FastAPI (controller and worker APIs), httpx (worker calls), and Unix-socket transport to Cloud Hypervisor. No shell=True anywhere.

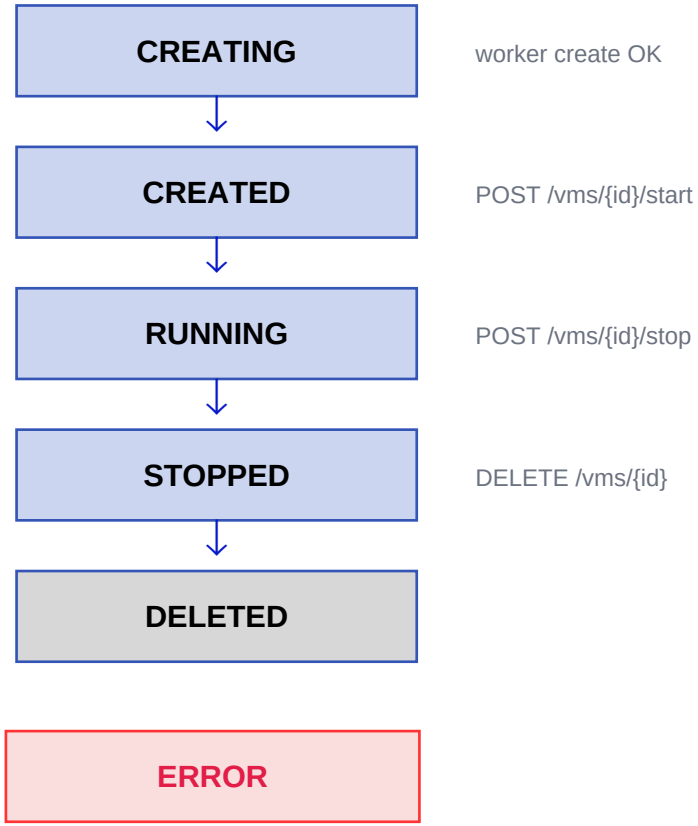
Controller: authentication, validation (1–4 vCPU, 128–2048 MB), placement, VM records, heartbeats, and retries with backoff.

Worker: per-VM disk copy, TAP/IP allocation, spawn Cloud Hypervisor in a cgroup v2 scope and its own process group, CH API calls, and reconciler.

Placement: choose a healthy host with the most free memory that fits the request plus a 512 MB reserve; tie-break by fewer VMs. If none fits, reject with 507.

Deterministic lifecycle: allow only legal state transitions; invalid transitions return HTTP 409. Operations are idempotent.

VM lifecycle state machine



Any failed step: rollback and reclaim resources.
Idempotent stop on STOPPED returns OK.
Invalid transitions (e.g. start on DELETED) return 409.

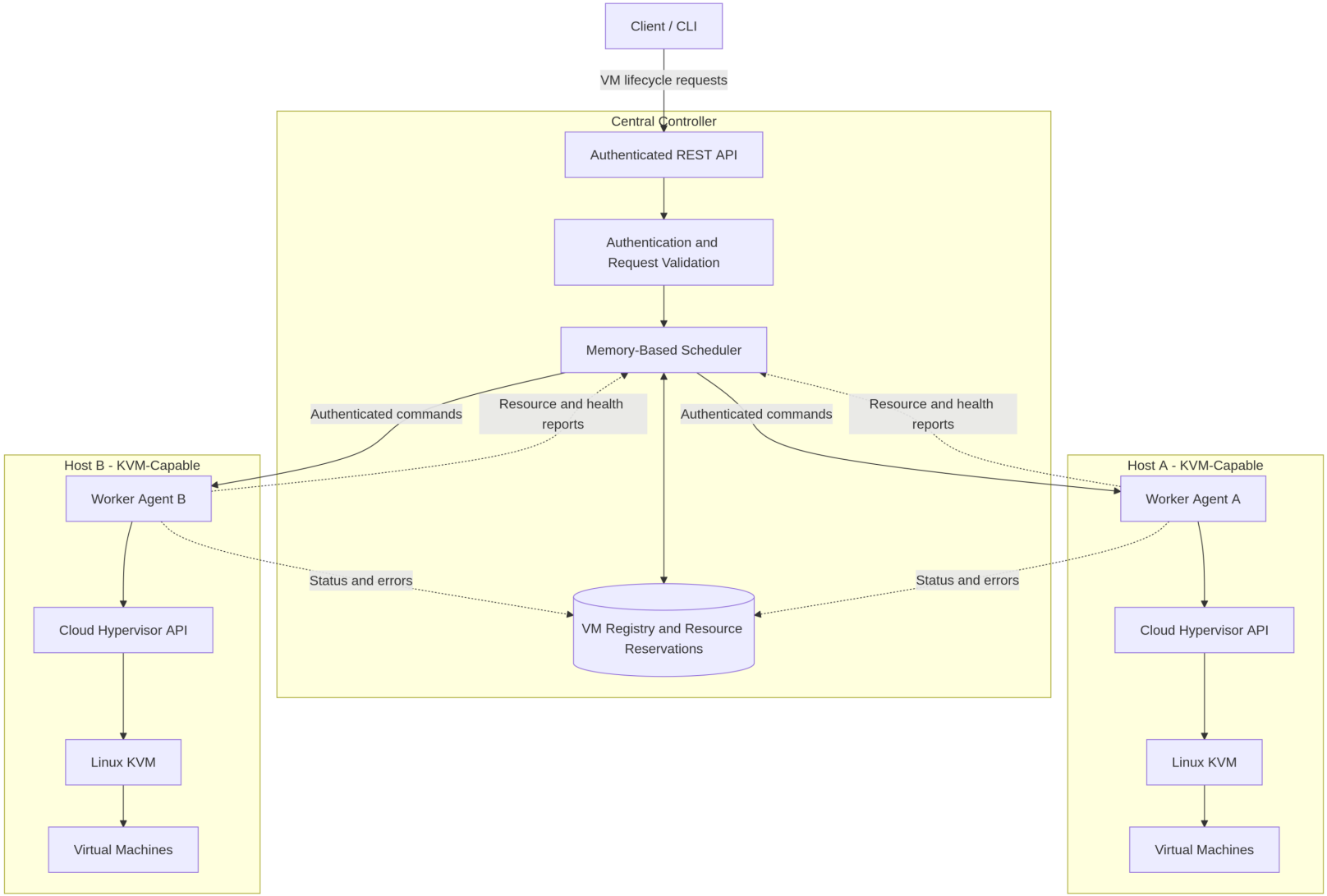
Optimization Proposed

Optimization strategies for efficient placement, lifecycle management, and resource control

Optimization	Proposed approach	Expected benefit
Memory-aware scheduling	Choose a healthy host with enough free memory for the VM request plus a 512 MB reserve. Break ties by selecting the host running fewer VMs.	Reduces placement failures and helps avoid memory overcommit.
Reservation before dispatch	Record the requested memory reservation before sending the create request to a worker.	Reduces the risk that concurrent requests allocate the same available memory.
Idempotent lifecycle APIs	Make safe repeated operations predictable; reject invalid VM state transitions with HTTP 409.	Prevents inconsistent state and makes client retries safer.
Process and resource isolation	Run each hypervisor process in its own process group and cgroup v2 scope, with configured vCPU and memory limits.	Contains process failures and enforces per-VM resource limits.

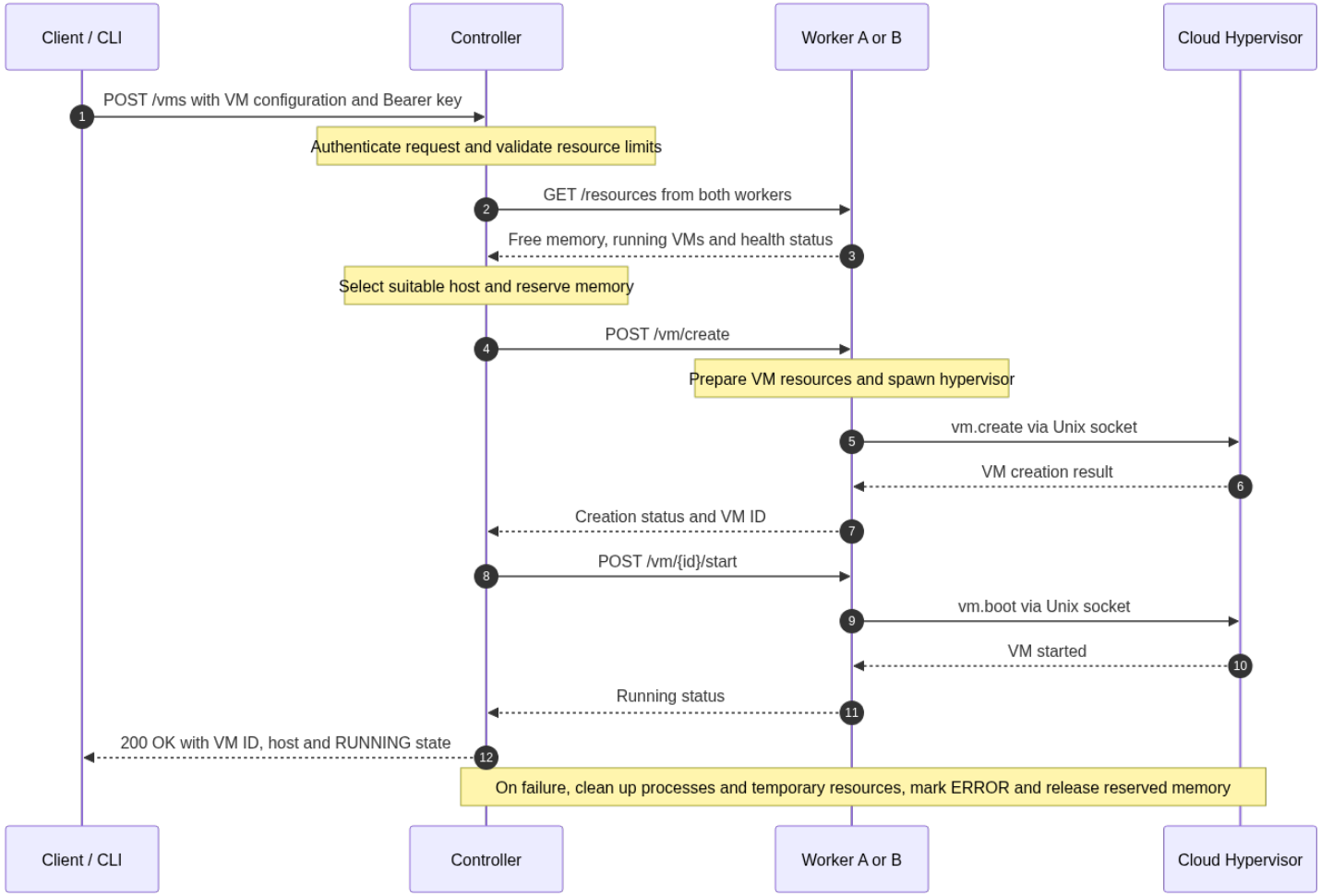
Solution Architecture & Design

Central controller, authenticated worker agents, and two KVM-capable hosts



Data Flow Diagram

VM creation and startup sequence across the controller, worker agent, and Cloud Hypervisor



Flow: authenticated request → resource discovery and host selection → VM creation/startup → running status; failures trigger cleanup and release of reserved memory.

Deployment Timeline

Milestones M0–M7 and delivery methodology

Milestone	Scope	Exit criterion	Owner
M0 Setup	Repo and access to two KVM hosts; verify `/dev/kvm`; install Cloud Hypervisor; boot one VM manually.	Manual VM boot works; environment notes recorded.	All
M1 Worker core	Launch, boot, and stop a VM through the Cloud Hypervisor API socket from Python.	Scripted lifecycle succeeds on one host.	Worker lead
M2 Worker API	Worker REST API, shared-secret authentication, rollback, and process-group teardown.	Lifecycle test leaves no resources behind.	Worker lead
M3 Controller	FastAPI endpoints, SQLite VM records, Bearer authentication, and VM state machine.	Create/start/stop/list/delete work via controller.	Controller lead
M4 Placement + limits	Resource polling, memory placement with locking/reservations, validation, and cgroup v2 limits.	Full Host A places on B; both full rejects request.	Controller + worker
M5 Failure handling	Reconciliation, heartbeats, unhealthy-worker marking, bounded retries, and idempotency.	Worker-down and hypervisor-failure tests recover or fail cleanly.	Controller + worker
M6 Tests + leak check	Unit, lifecycle, and failure-injection tests; check processes, sockets, and TAP/IP leftovers.	Tests pass; no leaks detected.	All
M7 Report + demo	Report, one-page pseudocode, 5–7 minute demo rehearsal, and repository handover.	Submission package and demo ready.	All

Methodology: M0 manual-first learning; separate controller/agent feature branches with peer review; early integration tests for API failures and memory constraints; regular syncs on the controller–worker API contract.

Bibliography

References cited on the Literature & RFC Review slide

[1] **Cloud Hypervisor Project.** “Cloud Hypervisor API.” Cloud Hypervisor Documentation. Accessed October 15, 2026.
https://github.com/cloud-hypervisor/cloud-hypervisor/blob/main/docs/api.md

[2] **Linux Kernel Documentation.** “The Definitive KVM (Kernel-based Virtual Machine) API Documentation.”
https://www.kernel.org/doc/html/latest/virt/kvm/api.html

[3] **Cloud Hypervisor Project.** “Cloud Hypervisor.” Source repository and project documentation. Accessed October 15, 2026.
https://github.com/cloud-hypervisor/cloud-hypervisor

[4] **Jones, M., and D. Hardt.** “The OAuth 2.0 Authorization Framework: Bearer Token Usage.” RFC 6750, Internet Engineering Task Force, October 2012.
https://www.rfc-editor.org/rfc/rfc6750.html

Citation key: [1] Cloud Hypervisor lifecycle API · [2] KVM host virtualization interface · [3] project implementation and operating context · [4] bearer-token HTTP authorization.

Thank You

PS-023 · Minimal Two-Node Cloud Hypervisor VM Controller

Team Disciples of GO

Joy Samadder Faculty Member · Assistant Professor	**Hrittik Biswas** Student · Worker Agent Lead	**Reek Saha** Student · Controller Lead

AIORI 3 · Team Disciples of GO